

# UNIFORM COST SEARCH

Artificial Intelligence / Python Practical

|                        |                                                 |
|------------------------|-------------------------------------------------|
| Experiment             | 05                                              |
| Title                  | Implementation of Uniform Cost Search in Python |
| Language               | Python                                          |
| Search Type            | Uninformed Search (Cost-Based)                  |
| Primary Data Structure | Priority Queue (Min-Heap)                       |

## 1. Aim

To write and execute a Python program that implements the Uniform Cost Search algorithm to find the lowest-cost path in a weighted graph.

## 2. Objective

The objective is to understand how an uninformed search algorithm can use edge costs to find the cheapest path from a start node to a goal node, by always expanding the node with the lowest cumulative path cost  $g(n)$ .

## 3. Theory

Uniform Cost Search (UCS) is an uninformed search technique. Unlike Breadth-First Search, which counts the number of edges, UCS always expands the node with the lowest cumulative path cost from the start node.

A priority queue (min-heap) is used to keep the frontier nodes ordered by their cumulative cost  $g(n)$ . A visited set is maintained so that a node is not expanded more than once.

The goal test is applied only when a node is removed from the priority queue, not when it is added. This guarantees that the first time the goal is removed, the path found is the lowest-cost path, provided every edge cost is non-negative.

## 4. Graph Used in the Practical

For this practical, the following weighted graph is used:

| Node | Neighbours (edge cost) |
|------|------------------------|
| A    | B (1), C (4)           |
| B    | C (2), D (5)           |
| C    | D (1), E (6)           |
| D    | G (3)                  |
| E    | G (1)                  |

|   |   |
|---|---|
| G | — |
|---|---|

**Start node:** A    **Goal node:** G

Graph relationships:  $A \rightarrow B$  (1),  $C$  (4);  $B \rightarrow C$  (2),  $D$  (5);  $C \rightarrow D$  (1),  $E$  (6);  $D \rightarrow G$  (3);  $E \rightarrow G$  (1).

## 5. Algorithm

- Initialize a priority queue with the start node and cost 0.
- Create an empty visited set.
- Remove the entry with the smallest cumulative cost from the priority queue.
- If the node has already been visited, skip it.
- Mark the node as visited.
- If the current node is the goal, stop the search — this path is the lowest-cost path.
- Otherwise, insert its unvisited neighbours into the priority queue with cost = current cost + edge cost.
- Repeat until the goal is found or the priority queue becomes empty.

## 6. Python Program

```
import heapq

graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('C', 2), ('D', 5)],
    'C': [('D', 1), ('E', 6)],
    'D': [('G', 3)],
    'E': [('G', 1)],
    'G': []
}

def uniform_cost_search(graph, start, goal):
    priority_queue = [(0, start, [start])]
    visited = set()

    while priority_queue:
        cost, node, path = heapq.heappop(priority_queue)

        if node in visited:
            continue
        visited.add(node)

        if node == goal:
            return path, cost

        for neighbour, edge_cost in graph[node]:
            if neighbour not in visited:
                heapq.heappush(
                    priority_queue,
```

```
        (cost + edge_cost, neighbour, path + [neighbour])
    )

    return None, None

start = 'A'
goal = 'G'
path, total_cost = uniform_cost_search(graph, start, goal)

print("Uniform Cost Search Path:")
print(" -> ".join(path))
print("Total Cost:", total_cost)
```

## 7. Expected Output

```
Uniform Cost Search Path:
A -> B -> C -> D -> G
Total Cost: 7
```

## 8. Step-by-Step Execution

| Step | Removed Node | Cost | Action                    |
|------|--------------|------|---------------------------|
| 1    | A            | 0    | Add B (1), C (4)          |
| 2    | B            | 1    | Add C (3), D (6)          |
| 3    | C            | 3    | Add D (4), E (9)          |
| 4    | C (via A)    | 4    | Already visited — skipped |
| 5    | D            | 4    | Add G (7)                 |
| 6    | D (via B)    | 6    | Already visited — skipped |
| 7    | G            | 7    | Goal found                |

Note: the fewest-edge path is  $A \rightarrow B \rightarrow D \rightarrow G$  (3 edges, cost 9), but UCS returns  $A \rightarrow B \rightarrow C \rightarrow D \rightarrow G$  (4 edges, cost 7) because it is cheaper overall. This is the key case where BFS and UCS disagree.

## 9. Time and Space Complexity

For a heap-based implementation, the time complexity is  $O((V + E) \log V)$ , where  $V$  is the number of vertices and  $E$  is the number of edges. In the general theoretical form, UCS complexity is expressed as  $O(b^{(1 + C^*/\epsilon)})$ , where  $b$  is the branching factor,  $C^*$  is the cost of the optimal solution, and  $\epsilon$  is the smallest edge cost. The space requirement is  $O(V)$  for the visited set and priority queue.

## 10. Advantages

- Finds the lowest-cost path when all edge costs are non-negative.
- Complete — finds a solution if one exists.
- Works correctly on weighted graphs, unlike BFS.
- Simple to implement using a priority queue.

## 11. Disadvantages

- Uses no heuristic, so it can explore many unnecessary nodes.
- Can require significant memory for large graphs.
- Slower than informed methods such as A\*, which use goal-direction.
- Does not work correctly with negative edge costs.

## 12. Applications

- Route and road-network path finding
- Network routing
- Cost-based planning problems
- Any search problem where actions have different costs

## 13. Uniform Cost Search vs BFS vs A\*

| Feature           | BFS                      | UCS                        | A*                        |
|-------------------|--------------------------|----------------------------|---------------------------|
| Guidance          | Level/depth              | Cumulative cost $g(n)$     | $g(n) + h(n)$             |
| Typical structure | FIFO queue               | Priority queue             | Priority queue            |
| Uses edge cost?   | No                       | Yes                        | Yes                       |
| Uses heuristic?   | No                       | No                         | Yes                       |
| Optimality        | Unweighted shortest path | Lowest cost (non-negative) | Under suitable conditions |

## 14. Viva Questions and Answers

### 1. What is Uniform Cost Search?

It is an uninformed search algorithm that always expands the node with the lowest cumulative path cost.

### 2. Which data structure is commonly used?

A priority queue (min-heap), ordered by cumulative cost.

### 3. What is $g(n)$ ?

The actual cost of the path from the start node to node  $n$ .

### 4. Does UCS use a heuristic?

No. It uses only the cost accumulated so far.

### 5. Is UCS optimal?

Yes, when all edge costs are non-negative.

### 6. What is the difference between UCS and BFS?

BFS finds the path with the fewest edges; UCS finds the path with the lowest total cost.

### 7. What is the difference between UCS and A\*?

UCS uses  $g(n)$  only, while A\* uses  $g(n) + h(n)$ .

### 8. When is the goal tested in UCS, and why?

When the node is removed from the priority queue, because that is when its cost is guaranteed to be lowest.

## 15. Conclusion

Uniform Cost Search was implemented in Python using a priority queue ordered by cumulative path cost. The experiment demonstrates that UCS finds the lowest-cost path,  $A \rightarrow B \rightarrow C \rightarrow D \rightarrow G$  with total cost 7, even though a path with fewer edges exists for the given graph.

## 16. Practical Record Note

This document is a clean, portfolio-ready practical record template. If your college-issued practical sheet uses a different graph, program, or output, those source-specific details should replace the corresponding sections before submission.